As I have already mentioned, *Graph_5* contains a Eulerian cycle. Let us modify the program '*graph5.sagews*' to check for a Eulerian cycle. Upon execution of the modified version of the program '*graph5.sagews*', we get the following output displayed on the screen.

```
[(0, 7), (7, 6), (6, 5), (5, 4), (4,
3), (3, 5), (5, 7), (7, 1), (1, 3), (3,
2), (2, 1), (1, 0)]
```

It can be seen that the cycle 0—7—6—5—4—3—5—7—1—3—2—1—0 visits all the 12 edges in *Graph_5* exactly once. Thus, it is a Eulerian cycle.

Now, let us discuss Hamiltonian cycles (named after the Irish mathematician William Rowan Hamilton) in a graph. This is a cycle in a graph where every vertex is visited exactly once. Under what circumstances does a Hamiltonian cycle come in handy in a real-life situation? Let us again consider the roads in a city. Assume this time you are tasked with inspecting the traffic lights at every junction in a city. Unlike the last time, you don't have to visit every road in the city. Here, a Hamiltonian cycle becomes very useful. Ideally, you would only visit each junction in the city (each vertex in the graph) exactly once. Again, there is no guarantee that such a route exists. There are graphs where you need to visit certain vertices more than once. Let us first check *Graph_5* for a Hamiltonian cycle. Add the following line of code at the end of the program called '*graph5.sagews*'.

```
G.hamiltonian_
cycle(algorithm='backtrack')
```

The above line of code checks whether *Graph_5* has a Hamiltonian cycle or not. The method returns *True* if there exists a Hamiltonian cycle in *Graph_5* along with the actual Hamiltonian cycle vertices as a list, using a backtracking algorithm. It returns *False* if there exists no Hamiltonian cycle along with a list showing where the algorithm failed (in the sense that it recognised there exists no Hamiltonian cycle). Upon execution of the modified version of the program '*graph5.sagews*', we get the following output displayed on the screen.

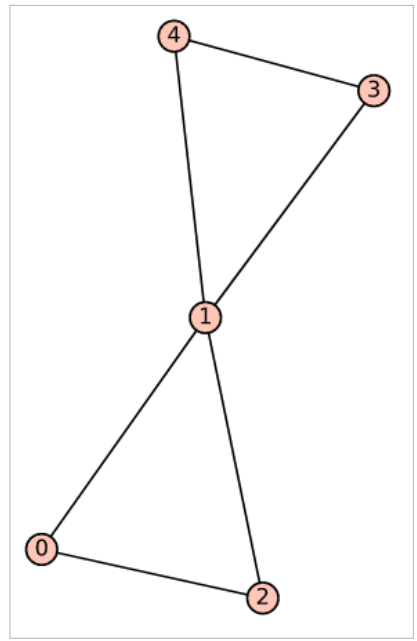

Figure 4: *Graph_6* generated by SageMath

```
TSP from : Graph on 8 vertices
(True, [4, 5, 6, 7, 0, 1, 2, 3])
```

It can be observed that the cycle 4—5—6—7—0—1—2—3—4 is a Hamiltonian cycle. Now, let us check whether *Graph_3* has a Hamiltonian cycle or not. Upon execution of the modified version of the program '*graph3.sagews*', we get the following output displayed on the screen.

```
TSP from : Graph on 6 vertices
(True, [5, 4, 1, 2, 3, 0])
```

It's clear that the cycle 5—4—1—2—3—0—5 is a Hamiltonian cycle. Recall that *Graph_3* does not have a Eulerian cycle. Modify the program '*graph4.sagews*' accordingly and verify that *Graph_4*, which does not have a Eulerian cycle, also has a Hamiltonian cycle. Thus, even if a graph does not have a Eulerian cycle, it still can contain a Hamiltonian cycle.

Now, we need to answer another question: If a graph has a Eulerian cycle, does it always have a Hamiltonian cycle? Let us modify Line 2 of the program '*graph3.sagews*' so that the adjacency matrix shown below is used to obtain a new program called '*graph6.sagews*'.

```
adj_matrix = [
  [0, 1, 1, 0, 0],
  [1, 0, 1, 1, 1],
  [1, 1, 0, 0, 0],
```

Upon execution of this program, we obtain the graph called *Graph_6*, as shown in Figure 4.

Now, add the following line of code at the end of the program called '*graph6.sagews*' to check whether *Graph_6* has a Eulerian cycle and a Hamiltonian cycle.

```
G.eulerian_circuit(labels=False)
G.hamiltonian_
cycle(algorithm='backtrack')
```

Upon execution of the modified version of the program '*graph6.sagews*', the following output is displayed on the screen.

```
[(0, 2), (2, 1), (1, 4), (4, 3), (3,
1), (1, 0)]
(False, [0, 2, 1, 4, 3])
```

The cycle 0—2—1—4—3—1—0 visits every edge of *Graph_6* exactly once and hence is a Eulerian cycle. Further, it has no Hamiltonian cycle. Thus, a graph not containing a Hamiltonian cycle can have a Eulerian cycle. As a further note, the problems of finding a Eulerian cycle and a Hamiltonian cycle form a very interesting class of problems in